Project Title: "OT-Sentinel: Offline Incident Response & Log Forensics Dashboard"

Here is a unique, custom project idea that is **Java-based**, **Web-based**, and **rarely found as a complete open-source package**:

1. The Real-World Problem It Solves:

In Operational Technology (OT) factories, devices (PLCs, RTUs) cannot always be connected to the internet for scanning. When a security breach is suspected, engineers manually collect log files (CSV/TXT). There is no simple, offline web tool to **upload, parse, visualize, and correlate** these logs to detect anomalies like "Repeated Failed Login Attempts" or "Unusual Command Sequences." Your project solves this by providing a **local, air-gapped web dashboard** for forensic analysis.

2. Why It's Not Easily Found:

- Most OT tools are expensive commercial software (e.g., Nozomi, Claroty).
- Open-source projects focus on *network scanning* (like Nmap wrappers), not *log forensics*.
- It requires a mix of **File Parsing**, **Data Visualization**, and **Rule-Based Detection**, which is complex enough to be a 3rd-year capstone but specific enough to be unique.

3. Core Features (MVP):

1. **Secure Local Login:** Spring Security with JWT.
2. **Log Ingestion Module:** Upload CSV/TXT logs from simulated PLCs/RTUs.
3. **Parsing Engine:** Java-based parser to extract timestamps, source IPs, command types, and status codes.
4. **Anomaly Detection Rule Engine:** A simple rule engine (e.g., "If >5 failed logins in 1 minute → Flag as Brute Force").

5. **Forensic Dashboard:** Charts showing "Command Frequency," "Failed vs. Successful Operations," and "Timeline of Events."
6. **Report Generation:** Export findings as a PDF incident report.

4. Tech Stack:

- **Backend:** Spring Boot (Java 17+)
- **Database:** H2 (for offline/portable use) or PostgreSQL
- **Frontend:** Thymeleaf (server-side rendering, easier to bundle) or React
- **Visualization:** Chart.js or ApexCharts (embedded in frontend)
- **PDF Generation:** iText or Apache PDFBox

How to Build It (Step-by-Step Guide)

Since there is no single GitHub link for this *exact* niche project, I will provide you with the **core code structure** and **key components** so you can build it quickly. This is better than a generic link because it's tailored to your OT background.

Step 1: Project Structure

```
com.ot.sentinel
├── config          # Security Config, WebConfig
├── controller      # LogController, DashboardController, AuthController
├── service         # LogParserService, AnomalyDetectionService, ReportService
├── repository      # LogEntryRepository, IncidentRepository
├── model           # LogEntry, Incident, User
└── util            # CsvParser, PdfGenerator
```

Step 2: Key Code Snippets

A. The Log Entry Model (`LogEntry.java`)

```java
@Entity
public class LogEntry {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String timestamp;
    private String sourceIp;
    private String commandType; // e.g., READ, WRITE, STOP
    private String status;      // SUCCESS, FAILURE
    private String deviceId;

    // Getters and Setters
}
```

B. The CSV Parser Service (`LogParserService.java`) *This is the "unique" part. It parses raw OT logs.*

```
@Service
public class LogParserService {

    @Autowired
    private LogEntryRepository logEntryRepository;

    public void parseAndSaveLogs(MultipartFile file) throws
IOException {
        List<LogEntry> entries = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new
InputStreamReader(file.getInputStream()))) {
            String line;
            while ((line = reader.readLine()) != null) {
                // Assuming CSV format: timestamp,sourceIp,
command,status,deviceId
                String[] values = line.split(",");
                if (values.length == 5) {
                    LogEntry entry = new LogEntry();
                    entry.setTimestamp(values[0]);
                    entry.setSourceIp(values[1]);
                    entry.setCommandType(values[2]);
                    entry.setStatus(values[3]);
                    entry.setDeviceId(values[4]);
                    entries.add(entry);
                }
            }
```

```
        }
        logEntryRepository.saveAll(entries);
    }
}
```

C. The Anomaly Detection Service (`AnomalyDetectionService.java`) *This is the "Brain" of your project.*

```
@Service
public class AnomalyDetectionService {

    @Autowired
    private LogEntryRepository logEntryRepository;

    public List<Incident> detectBruteForce() {
        // Find IPs with more than 5 failures in the last h
our
        List<LogEntry> failures = logEntryRepository.findBy
Status("FAILURE");
        Map<String, Integer> ipCount = new HashMap<>();

        for (LogEntry entry : failures) {
            ipCount.merge(entry.getSourceIp(), 1, Integer::
sum);
        }

        List<Incident> incidents = new ArrayList<>();
        for (Map.Entry<String, Integer> entry : ipCount.ent
rySet()) {
            if (entry.getValue() > 5) {
                Incident incident = new Incident();
                incident.setType("BRUTE_FORCE");
                incident.setDescription("Multiple failed at
tempts from IP: " + entry.getKey());
                incident.setSeverity("HIGH");
                incidents.add(incident);
            }
        }
```

```
        return incidents;
    }
}
```

D. The Dashboard Controller (`DashboardController.java`)

```
@Controller
public class DashboardController {

    @Autowired
    private LogEntryRepository logEntryRepository;

    @GetMapping("/dashboard")
    public String showDashboard(Model model) {
        model.addAttribute("totalLogs", logEntryRepository.
count());
        model.addAttribute("failureRate", calculateFailureR
ate());
        model.addAttribute("recentIncidents", anomalyDetect
ionService.detectBruteForce());
        return "dashboard"; // Thymeleaf template
    }
}
```

Step 3: How to Make It Look "Professional" & Unique

1. **Create Sample Data:** Generate a fake CSV file named `plc_logs.csv` with 100+ rows of data. Include some "attacks" (e.g., 10 failed logins in a row).
2. **Add a "Threat Map":** Use a simple JavaScript library to plot source IPs on a map (even if it's a static image with dots).
3. **Export to PDF:** Use **Apache PDFBox** to generate a one-page "Incident Report" with the detected anomalies.

Why This Will Get You Good Marks:

1. **Unique Domain:** It's not a generic CRUD app. It's a **Forensic Tool**.

2. **Real-World Relevance:** OT security is hot right now. Offline analysis is a real constraint in factories.
3. **Technical Depth:** Shows file handling, data parsing, algorithmic detection (brute force logic), and visualization.
4. **No API Limits:** Everything is local. You upload a file, and it processes it.

Where to Start?

1. Go to start.spring.io — Done
2. Add Dependencies: **Spring Web, Spring Security, Spring Data JPA, H2 Database, Thymeleaf**. — Done
3. Copy the code snippets above into your project.
4. Create a simple HTML/Thymeleaf page to upload the CSV.
5. Create a dashboard page to show the results.

This project is **custom-built for you** based on your OT background. It won't appear in a simple GitHub search for "Java student project" because it's a specialized niche.